NeuroRA Documentation

Release 2.0.0.1

Zitong Lu

Jul 23, 2026

Contents

A Python Toolbox of Representational Analysis from Multi-modal Neural Data **1**
Documentation of NeuroRA . . . 2
Overview . . . 2
Download . . . 2
API Reference . . . 3
RDM Calculation . . . 4
RDM Correlation . . . 7
RSA by RDM Correlation . . . 8
Cross-Temporal RDM Calculation . . . 14
Cross-Temporal RDM Correlation . . . 15
Cross-Temporal RSA Calculation . . . 16
Classification-Based Decoding . . . 18
Inter-Subject Correlation . . . 22
Statistical Analysis . . . 25
NIfTI Saving . . . 28
RSA Plotting . . . 30
Utilities . . . 42

A Python Toolbox of Representational Analysis from Multi-modal Neural Data

NeuroRA2 (an updated version of NeuroRA) is an easy-to-use toolbox based on Python, which can do some works about representational similarity analysis (RSA) among nearly all kinds of neural data, including behavioral, EEG, MEG, fNIRS, sEEG, ECoG, fMRI and some other neuroelectrophysiological data. In addition, users can do Inter-Subject Correlation (ISC), Classification-based EEG decoding and a novel cross-temporal RSA (CTRSA) on NeuroRA.

Author: Zitong Lu

Documentation of NeuroRA

Overview

Representational Similarity Analysis (RSA) has become a popular and effective method to measure the representation of multivariable neural activity in different modes.

NeuroRA is an easy-to-use toolbox based on Python, which can do some works about RSA among nearly all kinds of neural data, including behavioral, EEG, MEG, fNIRS, sEEG, ECoG, fMRI and some other neuroelectrophysiological data. In addition, users can do Neural Pattern Similarity (NPS), Spatiotemporal Pattern Similarity (STPS) & Inter-Subject Correlation (ISC) on NeuroRA.

NeuroRA includes some typical features below:

- Calculate the Neural Pattern Similarity (NPS)
- Calculate the Spatiotemporal Neural Pattern Similarity (STPS)
- Calculate the Inter-Subject Correlation (ISC)
- Calculate the Representational Dissimilarity Matrix (RDM)
- Calculate the Representational Similarity based on RDMs
- One-Step Realize Representational Similarity Analysis (RSA)
- Statistical Analysis
- Save the RSA result as a NIfTI file for fMRI
- Visualization for RSA results

Required Dependencies:

- Numpy: a fundamental package for scientific computing.
- SciPy: a package that provides many user-friendly and efficient numerical routines.
- Matplotlib: a Python 2D plotting library.
- NiBabel: a package prividing read +/- write access to some common medical and neuroimaging file formats.
- Nilearn: a Python module for fast and easy statistical learning on NeuroImaging data.
- MNE-Python: a Python software for exploring, visualizing, and analyzing human neurophysiological data.

Download

With Pip:

```
pip install neurora2
```

API Reference

The API reference is organized by NeuroRA module. Each module page starts with a function index followed by full API details.

RDM Calculation

neurora.rdm_cal

Function Index

- bhvRDM
- eegRDM
- eegRDM_bydecoding
- fmriRDM
- fmriRDM_roi

API Details

a module for calculating the RDM based on multimode neural data

neurora.rdm_cal.**bhvRDM**(*bhv_data*, *sub_opt=1*, *method='correlation'*)

Calculate RDMs for behavioral data.

Parameters

- **bhv_data** (*array_like, shape [n_cons, n_subs, n_trials]*) – Behavioral data.
- **sub_opt** (*{0, 1}, default=1*) – Whether to return subject-level RDMs or an averaged RDM.
  - 1: calculate one RDM for each subject.
  - 0: average trials and use subjects as features.
- **method** (*{"correlation", "amplitude"}, default="correlation"*) – Method used to calculate dissimilarities when sub_opt=0.

Returns

rdms – Behavioral RDMs.

If sub_opt=1, shape is [n_subs, n_cons, n_cons]. If sub_opt=0, shape is [n_cons, n_cons].

Return type

ndarray

neurora.rdm_cal.**eegRDM**(*EEG_data*, *chl_opt=0*, *time_opt=0*, *time_win=5*, *time_step=5*, *method='correlation'*)

Calculate RDMs based on EEG-like data.

Parameters

- **EEG_data** (*array_like, shape [n_cons, n_subs, n_trials, n_chls, n_ts]*) – EEG/MEG/fNIRS/ECoG/sEEG-like data.
- **chl_opt** (*{0, 1}, default=0*) – Whether to calculate RDMs separately for each channel.
  - 0: use all channels as features.
  - 1: calculate one RDM for each channel.
- **time_opt** (*{0, 1}, default=0*) – Whether to calculate RDMs separately for each time window.

- 0: use all time points as features.
- 1: calculate one RDM for each time window.

- **time_win** (*int, default=5*) – Time-window size in samples. Only used when time_opt=1.
- **time_step** (*int, default=5*) – Step size between adjacent time windows. Only used when time_opt=1.
- **method** (*{"correlation", "amplitude"}, default="correlation"*) – Method used to calculate dissimilarities.
  - correlation: 1 - Pearson correlation between condition patterns.
  - amplitude: absolute difference between mean feature amplitudes.

Returns

rdms – EEG-like RDMs.

If chl_opt=0 and time_opt=0: - shape [n_subs, n_cons, n_cons]

If chl_opt=0 and time_opt=1: - shape [n_subs, n_windows, n_cons, n_cons]

If chl_opt=1 and time_opt=0: - shape [n_subs, n_chls, n_cons, n_cons]

If chl_opt=1 and time_opt=1: - shape [n_subs, n_chls, n_windows, n_cons, n_cons]

Return type
ndarray

neurora.rdm_cal.**eegRDM_bydecoding**(*EEG_data*, *time_win=5*, *time_step=5*, *navg=5*, *time_opt='average'*, *nfolds=5*, *nrepeats=2*, *normalization=False*)

Calculate RDMs using classification-based neural decoding for EEG-like data.

Parameters

- **EEG_data** (*array_like, shape [n_cons, n_subs, n_trials, n_chls, n_ts]*) – EEG-like data.
- **time_win** (*int, default=5*) – Time-window size in samples.
- **time_step** (*int, default=5*) – Time-step size in samples.
- **navg** (*int, default=5*) – Number of trials used to average.
- **time_opt** (*{"average", "features"}, default="average"*) – Whether to average time points or use time points as features.
- **nfolds** (*int, default=5*) – Number of folds.
- **nrepeats** (*int, default=2*) – Number of repeated iterations.
- **normalization** (*bool, default=False*) – Whether to normalize data.

Returns
rdms – Decoding-based EEG-like RDMs.

Return type
ndarray, shape [n_subs, n_windows, n_cons, n_cons]

neurora.rdm_cal.**fmriRDM**(*fmri_data*, *ksize=(3, 3, 3)*, *strides=(1, 1, 1)*, *method='correlation'*)

Calculate searchlight RDMs based on fMRI data.

Parameters

- **fmri_data** (*array_like, shape [n_cons, n_subs, nx, ny, nz]*) – fMRI data.
- **ksize** (*tuple or list of int, default=(3, 3, 3)*) – Searchlight kernel size along x, y, and z axes.
- **strides** (*tuple or list of int, default=(1, 1, 1)*) – Searchlight strides along x, y, and z axes.
- **method** (*{"correlation", "amplitude"}, default="correlation"*) – Method used to calculate dissimilarities.
  - correlation: 1 - Pearson correlation between condition patterns.
  - amplitude: absolute difference between mean voxel amplitudes.

Returns
subrdms – fMRI searchlight RDMs.

Return type
ndarray, shape [n_subs, n_x, n_y, n_z, n_cons, n_cons]

Notes

A searchlight unit is skipped for all subjects if any voxel in that searchlight contains NaN for any condition or subject.

neurora.rdm_cal.**fmriRDM_roi**(*fmri_data*, *mask_data*, *method='correlation'*)

Calculate ROI RDMs based on fMRI data.

Parameters

- **fmri_data** (*array_like, shape [n_cons, n_subs, nx, ny, nz]*) – fMRI data.
- **mask_data** (*array_like, shape [nx, ny, nz]*) – ROI mask. Non-zero and finite voxels are included.
- **method** (*{"correlation", "amplitude"}, default="correlation"*) – Method used to calculate dissimilarities.
  - correlation: 1 - Pearson correlation between condition patterns.
  - amplitude: absolute difference between mean voxel amplitudes.

Returns
subrdms – fMRI ROI RDMs.

Return type
ndarray, shape [n_subs, n_cons, n_cons]

Notes

A voxel is included only if it is non-zero in the mask, finite in the mask, and finite for all conditions and subjects in fmri_data.

RDM Correlation

`neurora.rdm_corr`

Function Index

- `rdm_correlation_spearman`

API Details

a module for calculating the Similarity/Spearman Correlation Coefficient between two RDMs

`neurora.rdm_corr.`**`rdm_correlation_spearman`**(*rdm1*, *rdm2*)

Calculate the Spearman correlation between two RDMs.

Parameters

- **rdm1** (*array_like, shape [n_cons, n_cons]*) – The first RDM.
- **rdm2** (*array_like, shape [n_cons, n_cons]*) – The second RDM.

Returns

corr – Spearman correlation result.

- corr[0] : Spearman correlation coefficient
- corr[1] : p-value

Return type

ndarray, shape [2]

RSA by RDM Correlation

neurora.corr_cal_by_rdm

Function Index

- rdms_corr
- fmrirdms_corr
- rdms_partialcorr
- fmrirdms_partialcorr
- rdms_corr_perm
- fmrirdms_corr_perm
- rdms_partialcorr_perm
- fmrirdms_partialcorr_perm

API Details

a module for calculating the Similarity (Spearman Correlation Cosfficient) between model RDMs and neural RDMs

neurora.corr_cal_by_rdm.**fmrirdms_corr**(*model_rdms*, *fmri_rdms*, *parallel=False*, *n_jobs=-1*, *backend='loky'*)

Calculate Spearman correlations between multiple model RDMs and fMRI searchlight RDMs.

Parameters

- **model_rdms** (*array_like, shape [n_models, n_cons, n_cons]*) – Model RDMs.

  Each model RDM can be a behavioral RDM, a hypothesis-based coding model RDM, or a computational model RDM.

- **fmri_rdms** (*array_like, shape [n_x, n_y, n_z, n_cons, n_cons]*) – fMRI searchlight RDMs.

  n_x, n_y, and n_z represent the number of searchlight units along the x, y, and z axes.

- **parallel** (*bool, default=False*) – Whether to use joblib-based parallel computation across fMRI searchlight RDMs.

- **n_jobs** (*int, default=-1*) – Number of workers used when parallel=True.

  If n_jobs=-1, all available CPU cores will be used.

- **backend** (*{"threading", "loky"}, default="loky"*) – Joblib backend used when parallel=True.

  Use threading for shared-memory parallelism and lower memory overhead. Use loky for process-based parallelism when CPU-bound Python code is the main bottleneck.

Returns

corrs – Spearman correlation results between each fMRI searchlight RDM and each model RDM.

The last dimension contains:

- corrs[, 0] : Spearman r

- corrs[, 1] : p-value

Return type
ndarray, shape [n_x, n_y, n_z, n_models, 2]

neurora.corr_cal_by_rdm.**fmrirdms_corr_perm**(*model_rdms*, *fmri_rdms*, *n_permutation=1000*, *parallel=False*, *n_jobs=-1*, *backend='loky'*, *random_state=None*)

Calculate permutation-based Spearman correlations between model RDMs and fMRI searchlight RDMs.

The permutation is performed by shuffling the condition labels of each fMRI searchlight RDM.

Parameters

- **model_rdms** (*array_like, shape [n_models, n_cons, n_cons]*) – Model RDMs.
- **fmri_rdms** (*array_like, shape [n_x, n_y, n_z, n_cons, n_cons]*) – fMRI searchlight RDMs.
- **n_permutation** (*int, default=1000*) – Number of condition-label permutations.
- **parallel** (*bool, default=False*) – Whether to parallelize across fMRI searchlight units.
- **n_jobs** (*int, default=-1*) – Number of workers used when parallel=True.
- **backend** (*{"threading", "loky"}, default="loky"*) – Joblib backend used when parallel=True.
- **random_state** (*int or None, default=None*) – Random seed.

Returns
perm_corrs – Permutation Spearman correlation results.

Return type
ndarray, shape [n_x, n_y, n_z, n_permutation, n_models, 2]

neurora.corr_cal_by_rdm.**fmrirdms_partialcorr**(*model_rdms*, *fmri_rdms*, *parallel=False*, *n_jobs=-1*, *backend='loky'*)

Calculate Spearman partial correlations between multiple model RDMs and fMRI searchlight RDMs.

For each model RDM, the partial correlation is computed between the fMRI searchlight RDM and the current model RDM while controlling for all other model RDMs.

Parameters

- **model_rdms** (*array_like, shape [n_models, n_cons, n_cons]*) – Model RDMs.

  Each model RDM can be a behavioral RDM, a hypothesis-based coding model RDM, or a computational model RDM.

- **fmri_rdms** (*array_like, shape [n_x, n_y, n_z, n_cons, n_cons]*) – fMRI searchlight RDMs.

  n_x, n_y, and n_z represent the number of searchlight units along the x, y, and z axes.

- **parallel** (*bool, default=False*) – Whether to use joblib-based parallel computation across fMRI searchlight RDMs.

  Parallelization is performed across searchlight units.

- **n_jobs** (*int, default=-1*) – Number of workers used when parallel=True.

  If n_jobs=-1, all available CPU cores will be used.

- **backend** (*{"threading", "loky"}, default="loky"*) – Joblib backend used when parallel=True.

Returns

corrs – Spearman partial correlation results between each fMRI searchlight RDM and each model RDM while controlling for the other model RDMs.

The last dimension contains:

- corrs[, 0] : Spearman partial correlation coefficient
- corrs[, 1] : p-value

Return type
ndarray, shape [n_x, n_y, n_z, n_models, 2]

neurora.corr_cal_by_rdm.**fmrirdms_partialcorr_perm**(*model_rdms*, *fmri_rdms*, *n_permutation=1000*, *parallel=False*, *n_jobs=-1*, *backend='loky'*, *random_state=None*)

Calculate permutation-based Spearman partial correlations between model RDMs and fMRI searchlight RDMs.

The permutation is performed by shuffling the condition labels of each fMRI searchlight RDM.

Parameters

- **model_rdms** (*array_like, shape [n_models, n_cons, n_cons]*) – Model RDMs.
- **fmri_rdms** (*array_like, shape [n_x, n_y, n_z, n_cons, n_cons]*) – fMRI searchlight RDMs.
- **n_permutation** (*int, default=1000*) – Number of condition-label permutations.
- **parallel** (*bool, default=False*) – Whether to parallelize across fMRI searchlight units.
- **n_jobs** (*int, default=-1*) – Number of workers used when parallel=True.
- **backend** (*{"threading", "loky"}, default="loky"*) – Joblib backend used when parallel=True.
- **random_state** (*int or None, default=None*) – Random seed.

Returns
perm_corrs – Permutation Spearman partial correlation results.

Return type
ndarray, shape [n_x, n_y, n_z, n_permutation, n_models, 2]

neurora.corr_cal_by_rdm.**rdms_corr**(*model_rdms*, *eeg_rdms*, *parallel=False*, *n_jobs=-1*, *backend='loky'*)

Calculate Spearman correlations between multiple model RDMs and EEG-like RDMs.

Parameters

- **model_rdms** (*array_like, shape [n_models, n_cons, n_cons]*) – Model RDMs.

  Each model RDM can be a behavioral RDM, a hypothesis-based coding model RDM, or a computational model RDM.

- **eeg_rdms** (*array_like*) – EEG/MEG/fNIRS/ECoG/sEEG/electrophysiological RDMs.

  Supported shapes are:

  – [n_cons, n_cons]

  – [n1, n_cons, n_cons]

- [n1, n2, n_cons, n_cons]
- [n1, n2, n3, n_cons, n_cons]

The leading dimensions can represent time windows, channels, subjects, or other analysis dimensions.

- **parallel** (*bool, default=False*) – Whether to use joblib-based parallel computation across EEG-like RDMs.
- **n_jobs** (*int, default=-1*) – Number of workers used when parallel=True.

  If n_jobs=-1, all available CPU cores will be used.
- **backend** (*{"threading", "loky"}, default="loky"*) – Joblib backend used when parallel=True.

  Use threading for shared-memory parallelism and lower memory overhead. Use loky for process-based parallelism when CPU-bound Python code is the main bottleneck.

Returns

corrs – Spearman correlation results between each EEG-like RDM and each model RDM.

If eeg_rdms has shape [n_cons, n_cons], corrs has shape:

- [n_models, 2]

If eeg_rdms has shape [n1, n_cons, n_cons], corrs has shape:

- [n1, n_models, 2]

If eeg_rdms has shape [n1, n2, n_cons, n_cons], corrs has shape:

- [n1, n2, n_models, 2]

If eeg_rdms has shape [n1, n2, n3, n_cons, n_cons], corrs has shape:

- [n1, n2, n3, n_models, 2]

The last dimension contains:

- corrs[, 0] : Spearman r
- corrs[, 1] : p-value

Return type
ndarray

neurora.corr_cal_by_rdm.**rdms_corr_perm**(*model_rdms*, *eeg_rdms*, *n_permutation=1000*, *parallel=False*, *n_jobs=-1*, *backend='loky'*, *random_state=None*)

Calculate permutation-based Spearman correlations between model RDMs and EEG-like RDMs.

The permutation is performed by shuffling the condition labels of each EEG-like RDM.

Parameters

- **model_rdms** (*array_like, shape [n_models, n_cons, n_cons]*) – Model RDMs.
- **eeg_rdms** (*array_like, shape [..., n_cons, n_cons]*) – EEG-like RDMs.
- **n_permutation** (*int, default=1000*) – Number of condition-label permutations.
- **parallel** (*bool, default=False*) – Whether to parallelize across EEG-like target RDMs.
- **n_jobs** (*int, default=-1*) – Number of workers used when parallel=True.

- **backend** (*{"threading", "loky"}, default="loky"*) – Joblib backend used when parallel=True.
- **random_state** (*int or None, default=None*) – Random seed.

Returns

perm_corrs – Permutation Spearman correlation results.

Return type

ndarray, shape [, n_permutation, n_models, 2]

neurora.corr_cal_by_rdm.**rdms_partialcorr**(*model_rdms*, *eeg_rdms*, *parallel=False*, *n_jobs=-1*, *backend='loky'*)

Calculate Spearman partial correlations between multiple model RDMs and EEG-like RDMs.

For each model RDM, the partial correlation is computed between the EEG-like RDM and the current model RDM while controlling for all other model RDMs.

Parameters

- **model_rdms** (*array_like, shape [n_models, n_cons, n_cons]*) – Model RDMs.

  Each model RDM can be a behavioral RDM, a hypothesis-based coding model RDM, or a computational model RDM.

- **eeg_rdms** (*array_like*) – EEG/MEG/fNIRS/ECoG/sEEG/electrophysiological RDMs.

  Supported shapes are:

  – [n_cons, n_cons]

  – [n1, n_cons, n_cons]

  – [n1, n2, n_cons, n_cons]

  – [n1, n2, n3, n_cons, n_cons]

  The leading dimensions can represent time windows, channels, subjects, or other analysis dimensions.

- **parallel** (*bool, default=False*) – Whether to use joblib-based parallel computation across EEG-like RDMs.

  Parallelization is performed across target RDMs. For typical EEG-like inputs, this means parallelizing across time windows or across the combination of leading dimensions.

- **n_jobs** (*int, default=-1*) – Number of workers used when parallel=True.

  If n_jobs=-1, all available CPU cores will be used.

- **backend** (*{"threading", "loky"}, default="loky"*) – Joblib backend used when parallel=True.

Returns

corrs – Spearman partial correlation results between each EEG-like RDM and each model RDM while controlling for the other model RDMs.

If eeg_rdms has shape [n_cons, n_cons], corrs has shape:

- [n_models, 2]

If eeg_rdms has shape [n1, n_cons, n_cons], corrs has shape:

- [n1, n_models, 2]

If eeg_rdms has shape [n1, n2, n_cons, n_cons], corrs has shape:

- [n1, n2, n_models, 2]

If eeg_rdms has shape [n1, n2, n3, n_cons, n_cons], corrs has shape:

- [n1, n2, n3, n_models, 2]

The last dimension contains:

- corrs[, 0] : Spearman partial correlation coefficient
- corrs[, 1] : p-value

Return type
ndarray

neurora.corr_cal_by_rdm.**rdms_partialcorr_perm**(*model_rdms*, *eeg_rdms*, *n_permutation=1000*, *parallel=False*, *n_jobs=-1*, *backend='loky'*, *random_state=None*)

Calculate permutation-based Spearman partial correlations between model RDMs and EEG-like RDMs.

The permutation is performed by shuffling the condition labels of each EEG-like RDM.

Parameters

- **model_rdms** (*array_like, shape [n_models, n_cons, n_cons]*) – Model RDMs.
- **eeg_rdms** (*array_like, shape [..., n_cons, n_cons]*) – EEG-like RDMs.
- **n_permutation** (*int, default=1000*) – Number of condition-label permutations.
- **parallel** (*bool, default=False*) – Whether to parallelize across EEG-like target RDMs.
- **n_jobs** (*int, default=-1*) – Number of workers used when parallel=True.
- **backend** (*{"threading", "loky"}, default="loky"*) – Joblib backend used when parallel=True.
- **random_state** (*int or None, default=None*) – Random seed.

Returns
perm_corrs – Permutation Spearman partial correlation results.

Return type
ndarray, shape [, n_permutation, n_models, 2]

Cross-Temporal RDM Calculation

`neurora.ctrdm_cal`

Function Index

- `ctRDM`

API Details

A module for calculating cross-temporal RDMs based on EEG-like data.

`neurora.ctrdm_cal.`**`ctRDM`**(*data*, *chl_opt=0*, *time_win=5*, *time_step=5*)

Calculate cross-temporal RDMs for EEG-like data.

Parameters

- **`data`** (*`array_like, shape [n_cons, n_subs, n_chls, n_ts]`*) – EEG-like data.

  n_cons, n_subs, n_chls, and n_ts represent the number of conditions, subjects, channels, and time points, respectively.

- **`chl_opt`** (*`int, default=0`*) – Whether to calculate channel-specific CTRDMs.
  - 0 : calculate CTRDMs after combining channels and time points
  - 1 : calculate CTRDMs separately for each channel
- **`time_win`** (*`int, default=5`*) – Time-window size for calculating CTRDMs.
- **`time_step`** (*`int, default=5`*) – Step size between adjacent time windows.

Returns

ctrdms – Cross-temporal RDMs.

If chl_opt=0, the shape is:

[n_subs, n_windows, n_windows, n_cons, n_cons]

If chl_opt=1, the shape is:

[n_subs, n_chls, n_windows, n_windows, n_cons, n_cons]

Return type

ndarray

Notes

This function always returns subject-level CTRDMs.

If you want the group-averaged CTRDMs, use:

np.nanmean(ctrdms, axis=0)

Cross-Temporal RDM Correlation

`neurora.ctrdm_corr`

Function Index

- `ctrdm_correlation_spearman`

API Details

A module for calculating correlations between two cross-temporal RDMs.

`neurora.ctrdm_corr.`**`ctrdm_correlation_spearman`**(*ctrdm1*, *ctrdm2*)

Calculate the Spearman correlation between two CTRDMs.

Parameters

- **ctrdm1** (*array_like, shape [n_cons, n_cons]*) – First cross-temporal RDM.
- **ctrdm2** (*array_like, shape [n_cons, n_cons]*) – Second cross-temporal RDM.

Returns

corr – Spearman correlation result.

corr[0] is the correlation coefficient. corr[1] is the p-value.

Return type

ndarray, shape [2]

Notes

Unlike standard symmetric RDMs, CTRDMs are not assumed to be symmetric. Therefore, all off-diagonal elements are used, not just the upper triangle.

Cross-Temporal RSA Calculation

`neurora.ctcorr_cal`

Function Index

- `ctsim_cal`
- `ctsim_rdms_cal`
- `ctsim_ctrdms_cal`

API Details

A module for calculating cross-temporal similarities.

`neurora.ctcorr_cal.`**`ctsim_cal`**(*data1*, *data2*, *chl_opt=0*, *time_win=5*, *time_step=5*)

Calculate cross-temporal similarities between neural data under two conditions.

Parameters

- **`data1`** (*`array_like, shape [n_subs, n_chls, n_ts]`*) – EEG/MEG/fNIRS-like data under two conditions.
- **`data2`** (*`array_like, shape [n_subs, n_chls, n_ts]`*) – EEG/MEG/fNIRS-like data under two conditions.
- **`chl_opt`** (*`int, default=0`*) – Whether to calculate channel-specific cross-temporal similarities.
  - 0 : combine channels and time points as features
  - 1 : calculate similarities separately for each channel
- **`time_win`** (*`int, default=5`*) – Time-window size.
- **`time_step`** (*`int, default=5`*) – Step size between adjacent time windows.

Returns

ctsimilarities – Cross-temporal similarities.

If chl_opt=0, shape is:

[n_subs, n_windows, n_windows, 2]

If chl_opt=1, shape is:

[n_subs, n_chls, n_windows, n_windows, 2]

The last dimension contains [r, p].

Return type

ndarray

`neurora.ctcorr_cal.`**`ctsim_ctrdms_cal`**(*ctrdms*, *model_rdm*)

Calculate cross-temporal similarities between CTRDMs and a model RDM.

Parameters

- **ctrdms** (*array_like, shape [n_ts, n_ts, n_cons, n_cons] or [..., n_ts, n_ts, n_cons, n_cons]*) – Cross-temporal RDMs.

  The leading dimensions can represent subjects, channels, ROIs, etc.

- **model_rdm** (*array_like, shape [n_cons, n_cons]*) – Model RDM.

Returns

ctsimilarities – Cross-temporal similarities.

If ctrdms has shape [n_ts, n_ts, n_cons, n_cons], output shape is:

[n_ts, n_ts, 2]

If ctrdms has shape [, n_ts, n_ts, n_cons, n_cons], output shape is:

[, n_ts, n_ts, 2]

The last dimension contains [r, p].

Return type
ndarray

Notes

CTRDMs are not assumed to be symmetric. Therefore, all off-diagonal elements are used when correlating each CTRDM with the model RDM.

neurora.ctcorr_cal.**ctsim_rdms_cal**(*rdms*)

Calculate cross-temporal similarities based on temporal RDMs.

Parameters
rdms (*array_like, shape [n_ts, n_cons, n_cons] or [..., n_ts, n_cons, n_cons]*) – Temporal RDMs.

The leading dimensions can represent subjects, channels, ROIs, etc.

Returns

ctsimilarities – Cross-temporal similarities.

If rdms has shape [n_ts, n_cons, n_cons], output shape is:

[n_ts, n_ts, 2]

If rdms has shape [, n_ts, n_cons, n_cons], output shape is:

[, n_ts, n_ts, 2]

The last dimension contains [r, p].

Return type
ndarray

Classification-Based Decoding

`neurora.decoding`

Function Index

- `tbyt_decoding_kfold`
- `ct_decoding_kfold`
- `unidirectional_transfer_decoding`
- `bidirectional_transfer_decoding`

API Details

a module for classification-based neural decoding

`neurora.decoding.`**`bidirectional_transfer_decoding`**(*data1*, *labels1*, *data2*, *labels2*, *n=2*, *navg=5*, *time_opt='average'*, *time_win=5*, *time_step=5*, *n_iter=10*, *normalization=False*, *pca=False*, *pca_components=0.95*, *parallel=False*, *n_jobs=-1*, *backend='loky'*, *random_state=None*)

Conduct bidirectional transfer decoding for EEG-like data.

Parameters

- **data1** (*array_like, shape [n_subs, n_trials1, n_chls, n_ts1]*) – Data from condition 1.
- **labels1** (*array_like, shape [n_subs, n_trials1]*) – Labels for condition 1.
- **data2** (*array_like, shape [n_subs, n_trials2, n_chls, n_ts2]*) – Data from condition 2.
- **labels2** (*array_like, shape [n_subs, n_trials2]*) – Labels for condition 2.
- **n** (*int, default=2*) – Number of categories for classification.
- **navg** (*int, default=5*) – Number of trials averaged into each pseudo-trial.
- **time_opt** (*{"average", "features"}, default="average"*) – Strategy for handling time points within each time window.
- **time_win** (*int, default=5*) – Size of the time window in samples.
- **time_step** (*int, default=5*) – Step size between adjacent time windows in samples.
- **n_iter** (*int, default=10*) – Number of repeated random pseudo-trial averaging iterations.
- **normalization** (*bool, default=False*) – Whether to standardize features.
- **pca** (*bool, default=False*) – Whether to apply PCA.
- **pca_components** (*int or float, default=0.95*) – Number of PCA components, or explained variance ratio.
- **parallel** (*bool, default=False*) – Whether to parallelize across subjects using joblib.
- **n_jobs** (*int, default=-1*) – Number of workers used when parallel=True.

- **backend** (*{"loky", "threading"}, default="loky"*) – Joblib backend used when parallel=True.
- **random_state** (*int or None, default=None*) – Random seed.

Returns

- **con1_to_con2_accuracies** (*ndarray, shape [n_subs, n_train_windows1, n_test_windows2]*) – Transfer decoding accuracies from condition 1 to condition 2.
- **con2_to_con1_accuracies** (*ndarray, shape [n_subs, n_train_windows2, n_test_windows1]*) – Transfer decoding accuracies from condition 2 to condition 1.

neurora.decoding.**ct_decoding_kfold**(*data*, *labels*, *n=2*, *navg=5*, *time_opt='average'*, *time_win=5*, *time_step=5*, *nfolds=5*, *nrepeats=2*, *normalization=False*, *pca=False*, *pca_components=0.95*, *parallel=False*, *n_jobs=-1*, *backend='loky'*, *random_state=None*)

Conduct cross-temporal decoding for EEG-like data using cross-validation.

Parameters

- **data** (*array_like, shape [n_subs, n_trials, n_chls, n_ts]*) – EEG-like neural data.
- **labels** (*array_like, shape [n_subs, n_trials]*) – Trial labels for each subject.
- **n** (*int, default=2*) – Number of categories for classification.
- **navg** (*int, default=5*) – Number of trials averaged into each pseudo-trial.
- **time_opt** (*{"average", "features"}, default="average"*) – Strategy for handling time points within each time window.
- **time_win** (*int, default=5*) – Size of the time window in samples.
- **time_step** (*int, default=5*) – Step size between adjacent time windows in samples.
- **nfolds** (*int, default=5*) – Number of cross-validation folds.
- **nrepeats** (*int, default=2*) – Number of repeated random pseudo-trial averaging iterations.
- **normalization** (*bool, default=False*) – Whether to standardize features using the training data.
- **pca** (*bool, default=False*) – Whether to apply PCA after optional normalization.
- **pca_components** (*int or float, default=0.95*) – Number of PCA components, or explained variance ratio.
- **parallel** (*bool, default=False*) – Whether to parallelize across subjects using joblib.
- **n_jobs** (*int, default=-1*) – Number of workers used when parallel=True. If n_jobs=-1, all available CPU cores will be used.
- **backend** (*{"loky", "threading"}, default="loky"*) – Joblib backend used when parallel=True.
- **random_state** (*int or None, default=None*) – Random seed for reproducible pseudo-trial averaging and cross-validation.

Returns

accuracies – Cross-temporal decoding accuracies.

The second dimension is training time, and the third dimension is testing time.

Return type
ndarray, shape [n_subs, n_windows, n_windows]

neurora.decoding.**tbyt_decoding_kfold**(*data*, *labels*, *n=2*, *navg=5*, *time_opt='average'*, *time_win=5*, *time_step=5*, *nfolds=5*, *nrepeats=2*, *normalization=False*, *pca=False*, *pca_components=0.95*, *parallel=False*, *n_jobs=-1*, *backend='loky'*, *random_state=None*)

Conduct time-by-time decoding for EEG-like data using cross-validation.

Parameters

- **data** (*array_like, shape [n_subs, n_trials, n_chls, n_ts]*) – EEG-like neural data.
- **labels** (*array_like, shape [n_subs, n_trials]*) – Trial labels for each subject.
- **n** (*int, default=2*) – Number of categories for classification.
- **navg** (*int, default=5*) – Number of trials averaged into each pseudo-trial.
- **time_opt** (*{"average", "features"}, default="average"*) – Strategy for handling time points within each time window.
  - average: average time points within each window.
  - features: use all time points within each window as features.
- **time_win** (*int, default=5*) – Size of the time window in samples.
- **time_step** (*int, default=5*) – Step size between adjacent time windows in samples.
- **nfolds** (*int, default=5*) – Number of cross-validation folds.
- **nrepeats** (*int, default=2*) – Number of repeated random pseudo-trial averaging iterations.
- **normalization** (*bool, default=False*) – Whether to standardize features using the training data.
- **pca** (*bool, default=False*) – Whether to apply PCA after optional normalization.
- **pca_components** (*int or float, default=0.95*) – Number of PCA components, or explained variance ratio.
- **parallel** (*bool, default=False*) – Whether to parallelize across subjects using joblib.
- **n_jobs** (*int, default=-1*) – Number of workers used when parallel=True. If n_jobs=-1, all available CPU cores will be used.
- **backend** (*{"loky", "threading"}, default="loky"*) – Joblib backend used when parallel=True.
- **random_state** (*int or None, default=None*) – Random seed for reproducible pseudo-trial averaging and cross-validation.

Returns
accuracies – Time-by-time decoding accuracies.

Return type
ndarray, shape [n_subs, n_windows]

neurora.decoding.**unidirectional_transfer_decoding**(*data1*, *labels1*, *data2*, *labels2*, *n=2*, *navg=5*, *time_opt='average'*, *time_win=5*, *time_step=5*, *n_iter=10*, *normalization=False*, *pca=False*, *pca_components=0.95*, *parallel=False*, *n_jobs=-1*, *backend='loky'*, *random_state=None*)

Conduct unidirectional transfer decoding for EEG-like data.

The classifier is trained on data1 and tested on data2.

Parameters

- **data1** (*array_like, shape [n_subs, n_trials1, n_chls, n_ts1]*) – Training-condition EEG-like data.
- **labels1** (*array_like, shape [n_subs, n_trials1]*) – Labels for data1.
- **data2** (*array_like, shape [n_subs, n_trials2, n_chls, n_ts2]*) – Testing-condition EEG-like data.
- **labels2** (*array_like, shape [n_subs, n_trials2]*) – Labels for data2.
- **n** (*int, default=2*) – Number of categories for classification.
- **navg** (*int, default=5*) – Number of trials averaged into each pseudo-trial.
- **time_opt** (*{"average", "features"}, default="average"*) – Strategy for handling time points within each time window.
- **time_win** (*int, default=5*) – Size of the time window in samples.
- **time_step** (*int, default=5*) – Step size between adjacent time windows in samples.
- **n_iter** (*int, default=10*) – Number of repeated random pseudo-trial averaging iterations.
- **normalization** (*bool, default=False*) – Whether to standardize features using data1.
- **pca** (*bool, default=False*) – Whether to apply PCA after optional normalization.
- **pca_components** (*int or float, default=0.95*) – Number of PCA components, or explained variance ratio.
- **parallel** (*bool, default=False*) – Whether to parallelize across subjects using joblib.
- **n_jobs** (*int, default=-1*) – Number of workers used when parallel=True. If n_jobs=-1, all available CPU cores will be used.
- **backend** (*{"loky", "threading"}, default="loky"*) – Joblib backend used when parallel=True.
- **random_state** (*int or None, default=None*) – Random seed for reproducible pseudo-trial averaging.

Returns

accuracies – Unidirectional transfer decoding accuracies.

Return type

ndarray, shape [n_subs, n_train_windows, n_test_windows]

Inter-Subject Correlation

neurora.isc_cal

Function Index

- isc
- isc_fmri
- isc_fmri_roi

API Details

a module for calculating the inter-subject correlation based on neural data

neurora.isc_cal.**isc**(*data*, *time_win=5*, *time_step=5*, *parallel=False*, *n_jobs=-1*, *backend='loky'*)

Calculate inter-subject correlation (ISC) for EEG-like data.

Parameters

- **data** (*array_like, shape [n_subs, n_chls, n_ts]*) – EEG-like neural data.

  n_subs, n_chls, and n_ts represent the number of subjects, channels, and time points, respectively.

- **time_win** (*int, default=5*) – Time-window size in samples.
- **time_step** (*int, default=5*) – Step size between adjacent time windows.
- **parallel** (*bool, default=False*) – Whether to parallelize across subject pair Œ channel Œ time-window tasks.
- **n_jobs** (*int, default=-1*) – Number of workers used when parallel=True. If n_jobs=-1, all available CPU cores will be used.
- **backend** (*{"loky", "threading"}, default="loky"*) – Joblib backend used when parallel=True.

Returns

isc_results – ISC results.

The last dimension contains:

- isc_results[, 0] : r-value
- isc_results[, 1] : p-value

Return type

ndarray, shape [n_pairs, n_chls, n_windows, 2]

Notes

ISC is computed for each unique pair of subjects.

neurora.isc_cal.**isc_fmri**(*fmri_data*, *ksize=(3, 3, 3)*, *strides=(1, 1, 1)*, *parallel=False*, *n_jobs=-1*, *backend='loky'*)

Calculate inter-subject correlation (ISC) for fMRI data using searchlights.

Parameters

- **fmri_data** (*array_like, shape [n_ts, n_subs, nx, ny, nz]*) – fMRI data.

  n_ts, n_subs, nx, ny, and nz represent the number of time points, subjects, and spatial dimensions.

- **ksize** (*tuple or list of int, default=(3, 3, 3)*) – Searchlight kernel size along x, y, and z axes.

- **strides** (*tuple or list of int, default=(1, 1, 1)*) – Searchlight strides along x, y, and z axes.

- **parallel** (*bool, default=False*) – Whether to parallelize across time point Œ subject pair Œ searchlight tasks.

- **n_jobs** (*int, default=-1*) – Number of workers used when parallel=True. If n_jobs=-1, all available CPU cores will be used.

- **backend** (*{"loky", "threading"}, default="loky"*) – Joblib backend used when parallel=True.

Returns

isc_results – fMRI searchlight ISC results.

The last dimension contains:

- isc_results[, 0] : r-value
- isc_results[, 1] : p-value

Return type
ndarray, shape [n_ts, n_pairs, n_x, n_y, n_z, 2]

Notes

ISC is computed for each unique pair of subjects.

The NaN handling follows the original logic: for each time point, subject pair, and searchlight unit, the ISC is set to NaN if either subjects searchlight vector contains NaN.

neurora.isc_cal.**isc_fmri_roi**(*fmri_data*, *mask_data*, *parallel=False*, *n_jobs=-1*, *backend='loky'*)

Calculate inter-subject correlation (ISC) for fMRI data within an ROI.

Parameters

- **fmri_data** (*array_like, shape [n_ts, n_subs, nx, ny, nz]*) – fMRI data.
- **mask_data** (*array_like, shape [nx, ny, nz]*) – ROI mask. Non-zero and finite voxels are included.
- **parallel** (*bool, default=False*) – Whether to parallelize across time point Œ subject pair tasks.

- **n_jobs** (*int, default=-1*) – Number of workers used when parallel=True. If n_jobs=-1, all available CPU cores will be used.
- **backend** (*{"loky", "threading"}, default="loky"*) – Joblib backend used when parallel=True.

Returns

isc_results – fMRI ROI ISC results.

The last dimension contains:

- isc_results[, 0] : r-value
- isc_results[, 1] : p-value

Return type
ndarray, shape [n_ts, n_pairs, 2]

Notes

ISC is computed for each unique pair of subjects.

The ROI voxel selection follows the original logic: voxels are included if the mask is non-zero and finite. fMRI NaNs are handled at the time point Œ subject pair level, where the corresponding ISC value becomes NaN if either subjects ROI vector contains NaN.

Statistical Analysis

neurora.stats_cal

Function Index

- stats
- stats_fmri
- stats_iscfmri

API Details

a module for conducting the statistical analysis

neurora.stats_cal.**stats**(*corrs*, *alternative='greater'*)

Conduct statistical analysis for EEG-like correlation results.

Parameters

- **corrs** (*array_like, shape [n_subs, n_chls, n_ts]*) – Correlation coefficients.

  n_subs, n_chls, and n_ts represent the number of subjects, channels, and time points, respectively.

- **alternative** (*{"two-sided", "less", "greater"}, default="greater"*) – Alternative hypothesis for one-sample t-tests against zero.

Returns

stat_results – Statistical results.

The last dimension contains:

- stat_results[, 0] : t-value
- stat_results[, 1] : p-value

Return type

ndarray, shape [n_chls, n_ts, 2]

Notes

n_subs must be at least 6.

Fisher z transformation is always applied before the t-test.

This function can be used for subject-level EEG-like correlation results, such as NPS, ISC summaries, or EEG-like RDM correlations.

neurora.stats_cal.**stats_fmri**(*corrs*, *alternative='greater'*)

Conduct statistical analysis for fMRI searchlight correlation results.

Parameters

- **corrs** (*array_like, shape [n_subs, n_x, n_y, n_z]*) – Correlation coefficients.

  n_subs, n_x, n_y, and n_z represent the number of subjects and searchlight units along the x, y, and z axes.

- **alternative** (*{"two-sided", "less", "greater"}, default="greater"*) – Alternative hypothesis for one-sample t-tests against zero.

Returns

stat_results – Statistical results.

The last dimension contains:

- stat_results[, 0] : t-value
- stat_results[, 1] : p-value

Return type
ndarray, shape [n_x, n_y, n_z, 2]

Notes

n_subs must be at least 6.

Fisher z transformation is always applied before the t-test.

This function can be used for subject-level fMRI searchlight results, such as searchlight NPS or searchlight RDM correlations.

neurora.stats_cal.**stats_iscfmri**(*corrs*, *alternative='greater'*)

Conduct statistical analysis for fMRI ISC searchlight results.

Parameters

- **corrs** (*array_like, shape [n_ts, n_pairs, n_x, n_y, n_z]*) – ISC correlation coefficients.

  n_ts, n_pairs, n_x, n_y, and n_z represent the number of time points, subject pairs, and searchlight units along the x, y, and z axes.

- **alternative** (*{"two-sided", "less", "greater"}, default="greater"*) – Alternative hypothesis for one-sample t-tests against zero.

Returns

stat_results – Statistical results.

The last dimension contains:

- stat_results[, 0] : t-value
- stat_results[, 1] : p-value

Return type
ndarray, shape [n_ts, n_x, n_y, n_z, 2]

Notes

n_pairs must be at least 6.

Fisher z transformation is always applied before the t-test.

The second dimension of corrs is the subject-pair dimension. This function performs one-sample t-tests across subject pairs.

NIfTI Saving

neurora.nii_save

Function Index

- save_nii

API Details

a module for saving the RSA results in a .nii file for fMRI

neurora.nii_save.**save_nii**(*results*, *affine*, *filename=None*, *corr_mask='/Users/zitonglu/Documents/Projects/NeuroRA-master/neurora/template/HarvardOxford-cort-maxprob-thr0-1mm.nii.gz'*, *size=(60, 60, 60)*, *ksize=(3, 3, 3)*, *strides=(1, 1, 1)*, *p=0.05*, *value_threshold=None*, *correct_method=None*, *nvoxels_threshold=None*, *smooth=True*, *plotrlt=True*, *img_background=None*, *plot_type='r'*, *colorbar_label=None*)

Save fMRI RSA searchlight results as a NIfTI file.

Parameters

- **results** (*array_like, shape [n_x, n_y, n_z, 2]*) – Searchlight results.

  The last dimension contains:

  – results[, 0] : value map, such as r-values or t-values

  – results[, 1] : p-values

- **affine** (*array_like, shape [4, 4]*) – NIfTI affine matrix.

- **filename** (*str or None, default=None*) – Output NIfTI filename.

  If None, rsa_result.nii will be used.

- **corr_mask** (*str or None, default=get_HOcort()*) – Mask filename.

  If None, no mask will be applied.

- **size** (*tuple or list of int, default=(60, 60, 60)*) – Target image size [nx, ny, nz].

- **ksize** (*tuple or list of int, default=(3, 3, 3)*) – Searchlight kernel size [kx, ky, kz].

- **strides** (*tuple or list of int, default=(1, 1, 1)*) – Searchlight strides [sx, sy, sz].

- **p** (*float, default=0.05*) – P-value threshold.

- **value_threshold** (*float or None, default=None*) – Optional threshold for values.

  For correlation maps, this can be used as an r-value threshold. For statistical maps, this is usually None.

- **correct_method** (*{None, "FWE", "FDR"}, default=None*) – Multiple-comparison correction method.

  – None : no p-value correction

  - FWE : Bonferroni FWE correction
  - FDR : Benjamini-Hochberg FDR correction
- **nvoxels_threshold** (*int or None, default=None*) – Minimum cluster size in voxels.

  If not None, small clusters in the final volume will be removed using correct_by_threshold().
- **smooth** (*bool, default=True*) – Whether to smooth the output NIfTI image.
- **plotrlt** (*bool, default=True*) – Whether to plot the result automatically.
- **img_background** (*str or None, default=None*) – Background image filename used for plotting.
- **plot_type** (*{"r", "t"}, default="r"*) – Plot type passed to plot_brainrsa_rlts.
- **colorbar_label** (*str or None, default=None*) – Colorbar label. Reserved for future plot_brainrsa_rlts support.

Returns

volume – Saved result volume before smoothing.

Return type

ndarray, shape [nx, ny, nz]

RSA Plotting

neurora.rsa_plot

Function Index

- plot_rdm
- plot_tbyt_similarity
- plot_tbyt_decoding_acc
- plot_tbyt_similarity_multiple
- plot_tbyt_decoding_acc_multiple
- plot_ct_decoding_acc
- plot_brainrsa_regions
- plot_brainrsa_montage
- plot_brainrsa_glass
- plot_brainrsa_surface
- plot_brainrsa_rlts

API Details

a module for plotting the NeuroRA results

neurora.rsa_plot.**plot_brainrsa_glass**(*img*, *threshold=0*, *nvoxels_threshold=None*, *vmin=None*, *vmax=None*, *cmap='hot'*, *title='RSA Results'*, *colorbar_label=None*, *black_bg=True*, *draw_cross=True*, *connectivity=1*, *show=True*)

Plot a glass-brain projection of RSA results.

Parameters

- **img** (*str or Niimg-like object*) – NIfTI image of RSA results.
- **threshold** (*float or None, default=0*) – Display threshold passed to nilearn plotting.
- **nvoxels_threshold** (*int or None, default=None*) – Minimum cluster size in voxels.
- **vmin** (*float or None, default=None*) – Color limits.
- **vmax** (*float or None, default=None*) – Color limits.
- **cmap** (*str or matplotlib colormap, default="hot"*) – Colormap.
- **title** (*str or None, default="RSA Results"*) – Plot title.
- **colorbar_label** (*str or None, default=None*) – Colorbar label.
- **black_bg** (*bool, default=True*) – Whether to use a black background.
- **draw_cross** (*bool, default=True*) – Whether to draw crosshair.
- **connectivity** (*int, default=1*) – Connectivity used for cluster-size thresholding.
- **show** (*bool, default=True*) – Whether to show the plot immediately.

Returns
Nilearn display object.

Return type
display

neurora.rsa_plot.**plot_brainrsa_montage**(*img*, *threshold=0*, *nvoxels_threshold=None*, *cut_coords=(6, 6, 6)*, *background='/Users/zitonglu/Documents/Projects/NeuroRA-master/neurora/template/ch2bet.nii.gz'*, *vmin=None*, *vmax=None*, *cmap='hot'*, *title='RSA Results'*, *colorbar_label=None*, *draw_cross=True*, *connectivity=1*, *show=True*)

Plot RSA results across sagittal, coronal, and axial cuts.

Parameters

- **img** (*str or Niimg-like object*) – NIfTI image of RSA results.
- **threshold** (*float or None, default=0*) – Display threshold passed to nilearn plotting.
- **nvoxels_threshold** (*int or None, default=None*) – Minimum cluster size in voxels.
- **cut_coords** (*list or tuple, length 3, default=(6, 6, 6)*) – Cut coordinates for x, y, and z displays.

  Each element can be an int, float, list, tuple, or None. If an element is None or 0, that display direction is skipped.
- **background** (*Niimg-like object or str, default=get_bg_ch2bet()*) – Background image.
- **vmin** (*float or None, default=None*) – Color limits.
- **vmax** (*float or None, default=None*) – Color limits.
- **cmap** (*str or matplotlib colormap, default="hot"*) – Colormap.
- **title** (*str or None, default="RSA Results"*) – Base plot title.
- **colorbar_label** (*str or None, default=None*) – Colorbar label.
- **draw_cross** (*bool, default=True*) – Whether to draw crosshair.
- **connectivity** (*int, default=1*) – Connectivity used for cluster-size thresholding.
- **show** (*bool, default=True*) – Whether to show the plot immediately.

Returns
displays – Nilearn display objects for x, y, and z cuts.

Return type
dict

neurora.rsa_plot.**plot_brainrsa_regions**(*img*, *threshold=0*, *nvoxels_threshold=None*, *background='/Users/zitonglu/Documents/Projects/NeuroRA-master/neurora/template/ch2.nii.gz'*, *vmin=None*, *vmax=None*, *cmap='hot'*, *title='RSA Results'*, *colorbar_label=None*, *resampling_interpolation='continuous'*, *connectivity=1*, *show=True*)

Plot RSA-result regions using orthogonal brain cuts.

Parameters

- **img** (*str or Niimg-like object*) – NIfTI image of RSA results.
- **threshold** (*float or None, default=0*) – Display threshold passed to nilearn plotting.

- **nvoxels_threshold** (*int or None, default=None*) – Minimum cluster size in voxels.

  If None, no cluster-size thresholding is applied.
- **background** (*Niimg-like object or str, default=get_bg_ch2()*) – Background image.
- **vmin** (*float or None, default=None*) – Color limits.
- **vmax** (*float or None, default=None*) – Color limits.
- **cmap** (*str or matplotlib colormap, default="hot"*) – Colormap.
- **title** (*str or None, default="RSA Results"*) – Plot title.
- **colorbar_label** (*str or None, default=None*) – Colorbar label.
- **resampling_interpolation** (*str, default="continuous"*) – Resampling interpolation passed to nilearn.
- **connectivity** (*int, default=1*) – Connectivity used for cluster-size thresholding.
- **show** (*bool, default=True*) – Whether to show the plot immediately.

Returns
Nilearn display object.

Return type
display

neurora.rsa_plot.**plot_brainrsa_rlts**(*img*, *threshold=0.01*, *nvoxels_threshold=None*, *cut_coords=(6, 6, 6)*, *background=None*, *vmin=None*, *vmax=None*, *cmap='hot'*, *title='RSA Results'*, *colorbar_label=None*, *plot_regions=True*, *plot_montage=True*, *plot_glass=True*, *plot_surface=True*, *connectivity=1*, *show=True*)

Plot RSA brain results using multiple visualization modes.

Parameters

- **img** (*str or Niimg-like object*) – NIfTI image of RSA results.
- **threshold** (*float or None, default=0.01*) – Display threshold passed to nilearn plotting functions.
- **nvoxels_threshold** (*int or None, default=None*) – Minimum cluster size in voxels.
- **cut_coords** (*list or tuple, length 3, default=(6, 6, 6)*) – Cut coordinates for montage plotting.
- **background** (*Niimg-like object, str, or None, default=None*) – Background image.

  If None, default backgrounds are used.
- **vmin** (*float or None, default=None*) – Color limits.
- **vmax** (*float or None, default=None*) – Color limits.
- **cmap** (*str or matplotlib colormap, default="hot"*) – Colormap.
- **title** (*str or None, default="RSA Results"*) – Plot title.
- **colorbar_label** (*str or None, default=None*) – Colorbar label.
- **plot_regions** (*bool, default=True*) – Whether to plot orthogonal regions.
- **plot_montage** (*bool, default=True*) – Whether to plot montage cuts.

- **plot_glass** (*bool, default=True*) – Whether to plot glass brain.
- **plot_surface** (*bool, default=True*) – Whether to plot surface maps.
- **connectivity** (*int, default=1*) – Connectivity used for cluster-size thresholding.
- **show** (*bool, default=True*) – Whether to show the plots immediately.

Returns
displays – Dictionary of display objects.

Return type
dict

neurora.rsa_plot.**plot_brainrsa_surface**(*img*, *threshold=0.01*, *nvoxels_threshold=None*, *vmin=None*, *vmax=None*, *cmap='hot'*, *title='RSA Results'*, *colorbar_label=None*, *mesh='fsaverage5'*, *interpolation='linear'*, *views=('lateral', 'lateral', 'ventral', 'dorsal')*, *hemis=('left', 'right', 'both', 'both')*, *darkness=0.7*, *connectivity=1*, *show=True*)

Plot RSA results on fsaverage cortical surface.

Parameters

- **img** (*str or Niimg-like object*) – NIfTI image of RSA results.
- **threshold** (*float or None, default=0.01*) – Display threshold for surface plotting.
- **nvoxels_threshold** (*int or None, default=None*) – Minimum cluster size in voxels.
- **vmin** (*float or None, default=None*) – Color limits.
- **vmax** (*float or None, default=None*) – Color limits.
- **cmap** (*str or matplotlib colormap, default="hot"*) – Colormap.
- **title** (*str or None, default="RSA Results"*) – Base plot title.
- **colorbar_label** (*str or None, default=None*) – Colorbar label.
- **mesh** (*str, default="fsaverage5"*) – Fsaverage mesh density passed to nilearn.
- **interpolation** (*str, default="linear"*) – Interpolation method for vol_to_surf.
- **views** (*tuple of str, default=("lateral", "lateral", "ventral", "dorsal")*) – Surface views.
- **hemis** (*tuple of str, default=("left", "right", "both", "both")*) – Hemispheres to plot.
- **darkness** (*float, default=0.7*) – Background darkness.
- **connectivity** (*int, default=1*) – Connectivity used for cluster-size thresholding.
- **show** (*bool, default=True*) – Whether to show the plots immediately.

Returns
displays – Nilearn surface display objects.

Return type
list

neurora.rsa_plot.**plot_ct_decoding_acc**(*acc*, *start_timex=0*, *start_timey=0*, *time_intervalx=0.01*, *time_intervaly=0.01*, *smooth=True*, *smooth_window=5*, *chance=0.5*, *p=0.05*, *cbpt=True*, *clusterp=0.05*, *stats_timex=None*, *stats_timey=None*, *xlim=None*, *ylim=None*, *clim=(0.4, 0.8)*, *xlabel='Training Time (s)'*, *ylabel='Test Time (s)'*, *clabel='Decoding Accuracy'*, *figsize=(6.4, 4.8)*, *cmap='viridis'*, *contour_color='silver'*, *contour_linewidth=3*, *contour_linestyle='dashed'*, *ticksize=12*, *fontsize=16*, *title=None*, *title_fontsize=16*, *n_iter=1000*, *show=True*)

Plot cross-temporal decoding accuracies.

Parameters

- **acc** (*array_like, shape [n_subs, n_tsx, n_tsy]*) – Subject-level cross-temporal decoding accuracies.

  n_tsx is the number of training time points. n_tsy is the number of test time points.

- **start_timex** (*float, default=0*) – Time of the first training time point.
- **start_timey** (*float, default=0*) – Time of the first test time point.
- **time_intervalx** (*float, default=0.01*) – Training-time interval between adjacent time points.
- **time_intervaly** (*float, default=0.01*) – Test-time interval between adjacent time points.
- **smooth** (*bool, default=True*) – Whether to smooth each subjects cross-temporal matrix.
- **smooth_window** (*int, default=5*) – Approximate Gaussian smoothing window in time points.
- **chance** (*float, default=0.5*) – Chance level.
- **p** (*float, default=0.05*) – Point-wise or cluster-forming p-value threshold.
- **cbpt** (*bool, default=True*) – Whether to use cluster-based permutation test.
- **clusterp** (*float, default=0.05*) – Cluster-level p-value threshold.
- **stats_timex** (*tuple or list of float or None, default=None*) – Training-time window for statistical analysis.

  If None, the full training-time range is used.

- **stats_timey** (*tuple or list of float or None, default=None*) – Test-time window for statistical analysis.

  If None, the full test-time range is used.

- **xlim** (*tuple or None, default=None*) – Training-time axis limits.
- **ylim** (*tuple or None, default=None*) – Test-time axis limits.
- **clim** (*tuple, default=(0.4, 0.8)*) – Color limits.
- **xlabel** (*str, default="Training Time (s)"*) – X-axis label.
- **ylabel** (*str, default="Test Time (s)"*) – Y-axis label.
- **clabel** (*str, default="Decoding Accuracy"*) – Colorbar label.
- **figsize** (*tuple, default=(6.4, 4.8)*) – Figure size.
- **cmap** (*str or matplotlib colormap, default="viridis"*) – Colormap.

- **contour_color** (*matplotlib color, default="silver"*) – Color of significance contour.
- **contour_linewidth** (*int or float, default=3*) – Line width of significance contour.
- **contour_linestyle** (*str, default="dashed"*) – Line style of significance contour.
- **ticksize** (*int or float, default=12*) – Tick label size.
- **fontsize** (*int or float, default=16*) – Axis label size.
- **title** (*str or None, default=None*) – Figure title.
- **title_fontsize** (*int or float, default=16*) – Title font size.
- **n_iter** (*int, default=1000*) – Number of permutations for cluster-based permutation test.
- **show** (*bool, default=True*) – Whether to show the plot immediately.

Returns

ps – Binary significance map.

Return type

ndarray, shape [n_tsx, n_tsy]

neurora.rsa_plot.**plot_rdm**(*rdm*, *percentile=False*, *lim=(0, 1)*, *conditions=None*, *con_fontsize=12*, *cmap=None*, *title=None*, *title_fontsize=16*, *colorbar_label=None*, *colorbar_fontsize=16*, *show_values=False*, *value_fontsize=10*, *value_format='.4f'*, *value_color='blue'*, *figsize=None*, *show=True*)

Plot a representational dissimilarity matrix (RDM).

Parameters

- **rdm** (*array_like, shape [n_cons, n_cons]*) – Representational dissimilarity matrix.
- **percentile** (*bool, default=False*) – Whether to display RDM values as percentile ranks.
- **lim** (*tuple of float or None, default=(0, 1)*) – Color limits.

  If None, matplotlib determines the limits automatically.
- **conditions** (*list of str or None, default=None*) – Condition labels.
- **con_fontsize** (*int or float, default=12*) – Font size of condition labels.
- **cmap** (*matplotlib colormap or str or None, default=None*) – Colormap.

  If None, jet is used.
- **title** (*str or None, default=None*) – Figure title.
- **title_fontsize** (*int or float, default=16*) – Font size of the title.
- **colorbar_label** (*str or None, default=None*) – Colorbar label.

  If None, a default label is used.
- **colorbar_fontsize** (*int or float, default=16*) – Font size of colorbar tick labels and label.
- **show_values** (*bool, default=False*) – Whether to show numerical values inside RDM cells.
- **value_fontsize** (*int or float, default=10*) – Font size of displayed cell values.
- **value_format** (*str, default=".4f"*) – Format string for displayed values.

- **value_color** (*str, default="blue"*) – Color of displayed values.
- **figsize** (*tuple or None, default=None*) – Figure size.
- **show** (*bool, default=True*) – Whether to show the figure immediately.

Returns

- **fig** (*matplotlib.figure.Figure*) – Figure object.
- **ax** (*matplotlib.axes.Axes*) – Axes object.

neurora.rsa_plot.**plot_tbyt_decoding_acc**(*acc*, *start_time=0*, *time_interval=0.01*, *smooth=True*, *smooth_window=5*, *chance=0.5*, *p=0.05*, *cbpt=True*, *clusterp=0.05*, *stats_time=None*, *color='r'*, *xlim=None*, *ylim=(0.4, 0.8)*, *xlabel='Time (s)'*, *ylabel='Decoding Accuracy'*, *figsize=(6.4, 3.6)*, *x0=0*, *labelpad=0*, *ticksize=12*, *fontsize=16*, *markersize=2*, *title=None*, *title_fontsize=16*, *avgshow=False*, *n_iter=1000*, *show=True*)

Plot time-by-time decoding accuracies with statistical results.

Parameters

- **acc** (*array_like, shape [n_subs, n_ts]*) – Subject-level time-by-time decoding accuracies.
- **start_time** (*float, default=0*) – Time of the first data point.
- **time_interval** (*float, default=0.01*) – Time interval between adjacent data points.
- **smooth** (*bool, default=True*) – Whether to smooth each subjects time course.
- **smooth_window** (*int, default=5*) – Approximate smoothing window in time points.
- **chance** (*float, default=0.5*) – Chance level.
- **p** (*float, default=0.05*) – Point-wise or cluster-forming p-value threshold.
- **cbpt** (*bool, default=True*) – Whether to use cluster-based permutation test.
- **clusterp** (*float, default=0.05*) – Cluster-level p-value threshold.
- **stats_time** (*tuple or list of float or None, default=None*) – Time window for statistical analysis.

  If None, the full time range is used.
- **color** (*matplotlib color, default="r"*) – Plot color.
- **xlim** (*tuple or None, default=None*) – X-axis limits. If None, uses the full time range.
- **ylim** (*tuple, default=(0.4, 0.8)*) – Y-axis limits.
- **xlabel** (*str, default="Time (s)"*) – X-axis label.
- **ylabel** (*str, default="Decoding Accuracy"*) – Y-axis label.
- **figsize** (*tuple, default=(6.4, 3.6)*) – Figure size.
- **x0** (*float, default=0*) – X position of y-axis spine.
- **labelpad** (*int or float, default=0*) – Y-label padding.
- **ticksize** (*int or float, default=12*) – Tick label size.
- **fontsize** (*int or float, default=16*) – Axis label size.
- **markersize** (*int or float, default=2*) – Significant marker size.

- **title** (*str or None, default=None*) – Figure title.
- **title_fontsize** (*int or float, default=16*) – Title font size.
- **avgshow** (*bool, default=False*) – Whether to show the mean curve line.
- **n_iter** (*int, default=1000*) – Number of permutations for cluster-based permutation test.
- **show** (*bool, default=True*) – Whether to show the plot immediately.

Returns
ps – Binary significance vector.

Return type
ndarray, shape [n_ts]

neurora.rsa_plot.**plot_tbyt_decoding_acc_multiple**(*acc*, *start_time=0*, *time_interval=0.01*, *smooth=True*, *smooth_window=5*, *chance=0.5*, *p=0.05*, *cbpt=True*, *clusterp=0.05*, *stats_time=None*, *colors=None*, *condition_labels=None*, *xlim=None*, *ylim=(0.4, 0.8)*, *xlabel='Time (s)'*, *ylabel='Decoding Accuracy'*, *figsize=(6.4, 3.6)*, *x0=0*, *labelpad=0*, *ticksize=12*, *fontsize=16*, *title=None*, *title_fontsize=16*, *sig_linewidth=3*, *nonsig_linewidth=1*, *sem_alpha=0.2*, *nonsig_alpha=0.8*, *sig_alpha=1.0*, *nonsig_linestyle='--'*, *legend=True*, *n_iter=1000*, *show=True*)

Plot multiple time-by-time decoding accuracy results.

Parameters

- **acc** (*array_like, shape [n_conditions, n_subs, n_ts]*) – Multiple-condition subject-level decoding accuracy time courses.

  Shape should be [n_conditions, n_subs, n_timepoints].
- **start_time** (*float, default=0*) – Time of the first data point.
- **time_interval** (*float, default=0.01*) – Time interval between adjacent data points.
- **smooth** (*bool, default=True*) – Whether to smooth each subjects time course.
- **smooth_window** (*int, default=5*) – Approximate smoothing window in time points.
- **chance** (*float, default=0.5*) – Chance level.
- **p** (*float, default=0.05*) – Point-wise or cluster-forming p-value threshold.
- **cbpt** (*bool, default=True*) – Whether to use cluster-based permutation test.
- **clusterp** (*float, default=0.05*) – Cluster-level p-value threshold.
- **stats_time** (*tuple or list of float or None, default=None*) – Time window for statistical analysis.

  If None, the full time range is used.
- **colors** (*list or None, default=None*) – Colors for conditions.

  If None, matplotlib tab10 colors are used.
- **condition_labels** (*list of str or None, default=None*) – Condition labels for legend.

- **xlim** (*tuple or None, default=None*) – X-axis limits. If None, uses the full time range.
- **ylim** (*tuple, default=(0.4, 0.8)*) – Y-axis limits.
- **xlabel** (*str, default="Time (s)"*) – X-axis label.
- **ylabel** (*str, default="Decoding Accuracy"*) – Y-axis label.
- **figsize** (*tuple, default=(6.4, 3.6)*) – Figure size.
- **x0** (*float, default=0*) – X position of y-axis spine.
- **labelpad** (*int or float, default=0*) – Y-label padding.
- **ticksize** (*int or float, default=12*) – Tick label size.
- **fontsize** (*int or float, default=16*) – Axis label size.
- **title** (*str or None, default=None*) – Figure title.
- **title_fontsize** (*int or float, default=16*) – Title font size.
- **sig_linewidth** (*int or float, default=3*) – Line width for significant time points.
- **nonsig_linewidth** (*int or float, default=1*) – Line width for non-significant time points.
- **sem_alpha** (*float, default=0.2*) – Alpha for SEM shading.
- **nonsig_alpha** (*float, default=0.8*) – Alpha for non-significant dashed lines.
- **sig_alpha** (*float, default=1.0*) – Alpha for significant solid lines.
- **nonsig_linestyle** (*str, default="--"*) – Line style for non-significant time points.
- **legend** (*bool, default=True*) – Whether to show legend.
- **n_iter** (*int, default=1000*) – Number of permutations for cluster-based permutation test.
- **show** (*bool, default=True*) – Whether to show the plot immediately.

Returns

ps – Binary significance vectors.

Return type

ndarray, shape [n_conditions, n_ts]

neurora.rsa_plot.**plot_tbyt_similarity**(*similarities*, *start_time=0*, *time_interval=0.01*, *smooth=True*, *smooth_window=5*, *p=0.05*, *cbpt=True*, *clusterp=0.05*, *stats_time=None*, *color='r'*, *xlim=None*, *ylim=(-0.1, 0.8)*, *xlabel='Time (s)'*, *ylabel='Representational Similarity'*, *figsize=(6.4, 3.6)*, *x0=0*, *labelpad=0*, *ticksize=12*, *fontsize=16*, *markersize=2*, *title=None*, *title_fontsize=16*, *avgshow=False*, *n_iter=1000*, *show=True*)

Plot time-by-time representational similarities with statistical results.

Parameters

- **similarities** (*array_like, shape [n_subs, n_ts] or [n_subs, n_ts, 2]*) – Subject-level time-by-time similarities.

  If shape is [n_subs, n_ts, 2], only similarities[, 0] is used.
- **start_time** (*float, default=0*) – Time of the first data point.
- **time_interval** (*float, default=0.01*) – Time interval between adjacent data points.

- **smooth** (*bool, default=True*) – Whether to smooth each subjects time course.
- **smooth_window** (*int, default=5*) – Approximate smoothing window in time points.
- **p** (*float, default=0.05*) – Point-wise or cluster-forming p-value threshold.
- **cbpt** (*bool, default=True*) – Whether to use cluster-based permutation test.
- **clusterp** (*float, default=0.05*) – Cluster-level p-value threshold.
- **stats_time** (*tuple or list of float or None, default=None*) – Time window for statistical analysis.

  If None, the full time range is used.
- **color** (*matplotlib color, default="r"*) – Plot color.
- **xlim** (*tuple or None, default=None*) – X-axis limits. If None, uses the full time range.
- **ylim** (*tuple, default=(-0.1, 0.8)*) – Y-axis limits.
- **xlabel** (*str, default="Time (s)"*) – X-axis label.
- **ylabel** (*str, default="Representational Similarity"*) – Y-axis label.
- **figsize** (*tuple, default=(6.4, 3.6)*) – Figure size.
- **x0** (*float, default=0*) – X position of y-axis spine.
- **labelpad** (*int or float, default=0*) – Y-label padding.
- **ticksize** (*int or float, default=12*) – Tick label size.
- **fontsize** (*int or float, default=16*) – Axis label size.
- **markersize** (*int or float, default=2*) – Significant marker size.
- **title** (*str or None, default=None*) – Figure title.
- **title_fontsize** (*int or float, default=16*) – Title font size.
- **avgshow** (*bool, default=False*) – Whether to show the mean curve line.
- **n_iter** (*int, default=1000*) – Number of permutations for cluster-based permutation test.
- **show** (*bool, default=True*) – Whether to show the plot immediately.

Returns
ps – Binary significance vector.

Return type
ndarray, shape [n_ts]

neurora.rsa_plot.**plot_tbyt_similarity_multiple**(*similarities*, *start_time=0*, *time_interval=0.01*, *smooth=True*, *smooth_window=5*, *p=0.05*, *cbpt=True*, *clusterp=0.05*, *stats_time=None*, *colors=None*, *condition_labels=None*, *xlim=None*, *ylim=(-0.1, 0.8)*, *xlabel='Time (s)'*, *ylabel='Representational Similarity'*, *figsize=(6.4, 3.6)*, *x0=0*, *labelpad=0*, *ticksize=12*, *fontsize=16*, *title=None*, *title_fontsize=16*, *sig_linewidth=3*, *nonsig_linewidth=1*, *sem_alpha=0.2*, *nonsig_alpha=0.8*, *sig_alpha=1.0*, *nonsig_linestyle='--'*, *legend=True*, *n_iter=1000*, *show=True*)

Plot multiple time-by-time representational similarity results.

Parameters

- **similarities** (*array_like, shape [n_conditions, n_subs, n_ts]*) – Multiple-condition subject-level similarity time courses.

  Shape should be [n_conditions, n_subs, n_timepoints].

- **start_time** (*float, default=0*) – Time of the first data point.
- **time_interval** (*float, default=0.01*) – Time interval between adjacent data points.
- **smooth** (*bool, default=True*) – Whether to smooth each subjects time course.
- **smooth_window** (*int, default=5*) – Approximate smoothing window in time points.
- **p** (*float, default=0.05*) – Point-wise or cluster-forming p-value threshold.
- **cbpt** (*bool, default=True*) – Whether to use cluster-based permutation test.
- **clusterp** (*float, default=0.05*) – Cluster-level p-value threshold.
- **stats_time** (*tuple or list of float or None, default=None*) – Time window for statistical analysis.

  If None, the full time range is used.

- **colors** (*list or None, default=None*) – Colors for conditions.

  If None, matplotlib tab10 colors are used.

- **condition_labels** (*list of str or None, default=None*) – Condition labels for legend.
- **xlim** (*tuple or None, default=None*) – X-axis limits. If None, uses the full time range.
- **ylim** (*tuple, default=(-0.1, 0.8)*) – Y-axis limits.
- **xlabel** (*str, default="Time (s)"*) – X-axis label.
- **ylabel** (*str, default="Representational Similarity"*) – Y-axis label.
- **figsize** (*tuple, default=(6.4, 3.6)*) – Figure size.
- **x0** (*float, default=0*) – X position of y-axis spine.
- **labelpad** (*int or float, default=0*) – Y-label padding.
- **ticksize** (*int or float, default=12*) – Tick label size.
- **fontsize** (*int or float, default=16*) – Axis label size.
- **title** (*str or None, default=None*) – Figure title.
- **title_fontsize** (*int or float, default=16*) – Title font size.
- **sig_linewidth** (*int or float, default=3*) – Line width for significant time points.
- **nonsig_linewidth** (*int or float, default=1*) – Line width for non-significant time points.
- **sem_alpha** (*float, default=0.2*) – Alpha for SEM shading.
- **nonsig_alpha** (*float, default=0.8*) – Alpha for non-significant dashed lines.
- **sig_alpha** (*float, default=1.0*) – Alpha for significant solid lines.
- **nonsig_linestyle** (*str, default="--"*) – Line style for non-significant time points.
- **legend** (*bool, default=True*) – Whether to show legend.

- **n_iter** (*int, default=1000*) – Number of permutations for cluster-based permutation test.
- **show** (*bool, default=True*) – Whether to show the plot immediately.

Returns

ps – Binary significance vectors.

Return type

ndarray, shape [n_conditions, n_ts]

Utilities

neurora.stuff

Function Index

- limtozero
- get_affine
- correct_by_threshold
- fwe_correct
- fdr_correct
- get_bg_ch2
- get_bg_ch2bet
- get_HOcort
- datamask
- position_to_mni
- mniposition_to
- mask_to
- clusterbased_permutation_1d_1samp_1sided
- clusterbased_permutation_1d_1samp_2sided
- clusterbased_permutation_1d_1sided
- clusterbased_permutation_1d_2sided
- clusterbased_permutation_2d_1samp_1sided
- clusterbased_permutation_2d_1samp_2sided
- clusterbased_permutation_2d_1sided
- clusterbased_permutation_2d_2sided
- smooth_1d_gaussian
- smooth_2d_gaussian
- normalize_rdm

API Details

A module for simple but important utility functions.

neurora.stuff.**clusterbased_permutation_1d_1samp_1sided**(*results*, *level=0*, *p_threshold=0.05*, *clusterp_threshold=0.05*, *n_threshold=2*, *n_iter=1000*, *random_state=None*, *n_jobs=None*)

One-sample one-sided cluster-based permutation test for 1-D results.

Parameters

- **results** (*array_like, shape [n_subs, x]*) – Subject-level results.
- **level** (*float, default=0*) – Null-hypothesis level.
- **p_threshold** (*float, default=0.05*) – Cluster-forming p-value threshold.
- **clusterp_threshold** (*float, default=0.05*) – Cluster-level p-value threshold.
- **n_threshold** (*int, default=2*) – Minimum cluster size.
- **n_iter** (*int, default=1000*) – Number of permutations.
- **random_state** (*int or None, default=None*) – Random seed.
- **n_jobs** (*int or None, default=None*) – Number of parallel jobs for MNE.

Returns
ps – Binary significance vector.

Return type
ndarray, shape [x]

neurora.stuff.**clusterbased_permutation_1d_1samp_2sided**(*results*, *level=0*, *p_threshold=0.05*, *clusterp_threshold=0.05*, *n_threshold=2*, *n_iter=1000*, *random_state=None*, *n_jobs=None*)

One-sample two-sided cluster-based permutation test for 1-D results.

Returns

ps – Signed significance vector.

- 1 : significantly greater than level
- -1 : significantly less than level
- 0 : not significant

Return type
ndarray, shape [x]

neurora.stuff.**clusterbased_permutation_1d_1sided**(*results1*, *results2*, *p_threshold=0.05*, *clusterp_threshold=0.05*, *n_threshold=2*, *n_iter=1000*, *random_state=None*, *n_jobs=None*)

Paired one-sided cluster-based permutation test for 1-D results.

Tests whether results1 > results2.

neurora.stuff.**clusterbased_permutation_1d_2sided**(*results1*, *results2*, *p_threshold=0.05*, *clusterp_threshold=0.05*, *n_threshold=2*, *n_iter=1000*, *random_state=None*, *n_jobs=None*)

Paired two-sided cluster-based permutation test for 1-D results.

neurora.stuff.**clusterbased_permutation_2d_1samp_1sided**(*results*, *level=0*, *p_threshold=0.05*, *clusterp_threshold=0.05*, *n_threshold=4*, *n_iter=1000*, *random_state=None*, *n_jobs=None*)

One-sample one-sided cluster-based permutation test for 2-D results.

neurora.stuff.**clusterbased_permutation_2d_1samp_2sided**(*results*, *level=0*, *p_threshold=0.05*, *clusterp_threshold=0.05*, *n_threshold=4*, *n_iter=1000*, *random_state=None*, *n_jobs=None*)

One-sample two-sided cluster-based permutation test for 2-D results.

neurora.stuff.**clusterbased_permutation_2d_1sided**(*results1*, *results2*, *p_threshold=0.05*, *clusterp_threshold=0.05*, *n_threshold=4*, *n_iter=1000*, *random_state=None*, *n_jobs=None*)

Paired one-sided cluster-based permutation test for 2-D results.

Tests whether results1 > results2.

neurora.stuff.**clusterbased_permutation_2d_2sided**(*results1*, *results2*, *p_threshold=0.05*, *clusterp_threshold=0.05*, *n_threshold=4*, *n_iter=1000*, *random_state=None*, *n_jobs=None*)

Paired two-sided cluster-based permutation test for 2-D results.

neurora.stuff.**correct_by_threshold**(*img*, *threshold*, *connectivity=1*)

Correct fMRI RSA results by removing clusters smaller than a voxel threshold.

Parameters

- **img** (*array_like, shape [nx, ny, nz]*) – A 3-D fMRI RSA result map.
- **threshold** (*int*) – Minimum cluster size in voxels.

  Clusters with fewer than threshold non-zero voxels will be removed.
- **connectivity** (*{1, 2, 3}, default=1*) – Cluster connectivity.

Returns

corrected_img – Threshold-corrected image.

Return type

ndarray, shape [nx, ny, nz]

neurora.stuff.**datamask**(*fmri_data*, *mask_data*)

Filter a 3-D fMRI map by a 3-D ROI mask.

Parameters

- **fmri_data** (*array_like, shape [nx, ny, nz]*) – fMRI data.
- **mask_data** (*array_like, shape [nx, ny, nz]*) – ROI mask.

Returns

newfmri_data – Masked fMRI data. Voxels outside the mask are NaN.

Return type

ndarray, shape [nx, ny, nz]

neurora.stuff.**fdr_correct**(*p*, *p_threshold*)

FDR correction for fMRI RSA results using Benjamini-Hochberg correction.

Parameters

- **p** (*array_like, shape [nx, ny, nz]*) – A 3-D p-value map.
- **p_threshold** (*float*) – FDR-corrected p-value threshold.

Returns

fdrp – FDR-corrected p-value map.

Voxels whose corrected p-values do not survive p_threshold are NaN.

Return type
ndarray, shape [nx, ny, nz]

neurora.stuff.**fwe_correct**(*p*, *p_threshold*)

FWE correction for fMRI RSA results using Bonferroni correction.

Parameters

- **p** (*array_like, shape [nx, ny, nz]*) – A 3-D p-value map.
- **p_threshold** (*float*) – Corrected p-value threshold.

Returns

fwep – FWE-corrected p-value map.

Voxels whose corrected p-values do not survive p_threshold are NaN.

Return type
ndarray, shape [nx, ny, nz]

neurora.stuff.**get_HOcort**()

Get the absolute file path of HarvardOxford cortical atlas.

Returns
path – Absolute file path of HarvardOxford-cort-maxprob-thr0-1mm.nii.gz.

Return type
str

neurora.stuff.**get_affine**(*file_name*)

Get the affine matrix of a NIfTI image.

Parameters
file_name (*str*) – Filename of a sample fMRI image.

Returns
affine – Affine matrix of the image.

Return type
ndarray, shape [4, 4]

neurora.stuff.**get_bg_ch2**()

Get the absolute file path of ch2.nii.gz.

Returns
path – Absolute file path of ch2.nii.gz.

Return type
str

neurora.stuff.**get_bg_ch2bet**()

Get the absolute file path of ch2bet.nii.gz.

Returns
path – Absolute file path of ch2bet.nii.gz.

Return type
str

neurora.stuff.**limtozero**(*x*, *tol=1e-15*)

Set values very close to zero to exactly zero.

Parameters

- **x** (*float or array_like*) – Input value or array.
- **tol** (*float, default=1e-15*) – Absolute tolerance for treating values as zero.

Returns
out – Value with near-zero values replaced by zero.

Return type
float or ndarray

neurora.stuff.**mask_to**(*mask*, *size*, *affine*, *filename=None*)

Convert a mask image to a target data template.

Parameters

- **mask** (*str*) – File path of the mask image.
- **size** (*tuple or list of int, length 3*) – Target image size [nx, ny, nz].
- **affine** (*array_like, shape [4, 4]*) – Target affine matrix.
- **filename** (*str or None, default=None*) – Output filename. If None, newmask.nii will be used.

Returns
filename – Output filename of the generated mask image.

Return type
str

neurora.stuff.**mniposition_to**(*mnipoint*, *affine*)

Project MNI/world coordinates to voxel coordinates.

Parameters

- **mnipoint** (*array_like, shape [3]*) – MNI/world coordinate.
- **affine** (*array_like, shape [4, 4]*) – Affine matrix.

Returns
voxel_index – Voxel coordinate.

Return type
tuple of int

neurora.stuff.**normalize_rdm**(*rdm*, *method='minmax'*, *keep_diagonal=True*)

Normalize a representational dissimilarity matrix (RDM).

Parameters

- **rdm** (*array_like, shape [n_cons, n_cons] or [..., n_cons, n_cons]*) – Input RDM or a stack of RDMs.

  The last two dimensions should be square matrices.

- **method** (*{"minmax", "zscore"}, default="minmax"*) – Normalization method.

  – minmax: scale off-diagonal values to the range [0, 1]

  – zscore: standardize off-diagonal values to mean 0 and standard deviation 1

- **keep_diagonal** (*bool, default=True*) – Whether to keep the diagonal values unchanged.

  If True, diagonal values are copied from the original RDM. If False, diagonal values are set to 0.

Returns
normalized_rdm – Normalized RDM with the same shape as the input.

Return type
ndarray

Notes

Normalization is applied separately to each RDM when a stack of RDMs is provided.

Only off-diagonal values are used to estimate min/max or mean/std.

neurora.stuff.**position_to_mni**(*point*, *affine*)

Project voxel coordinates to MNI/world coordinates.

Parameters

- **point** (*array_like, shape [3]*) – Voxel coordinate.
- **affine** (*array_like, shape [4, 4]*) – Affine matrix.

Returns
newpoint – MNI/world coordinate.

Return type
ndarray, shape [3]

neurora.stuff.**smooth_1d_gaussian**(*data*, *smooth_window=5*)

Smooth 1-D time-series data using a Gaussian filter.

Parameters

- **data** (*array_like, shape [n_timepoints]*) – Input 1-D time-series data.
- **smooth_window** (*int, default=5*) – Approximate smoothing window in time points.

Returns
smoothed_data – Smoothed 1-D data.

Return type
ndarray, shape [n_timepoints]

neurora.stuff.**smooth_2d_gaussian**(*data*, *smooth_window=5*)

Smooth 2-D data using a Gaussian filter.

Parameters

- **data** (*array_like, shape [n_timepoints1, n_timepoints2]*) – Input 2-D data.
- **smooth_window** (*int, default=5*) – Approximate smoothing window in time points.

Returns
smoothed_data – Smoothed 2-D data.

Return type
ndarray, shape [n_timepoints1, n_timepoints2]